



TraceableHands

A Decentralised Charity Platform with Crowd-Audited Escrow

Donate · Track · Know

Course	Group	Semester
FT5004 Programming for Blockchain	Group 3 — Team 8888	AY 2025/2026 Sem 2

Agrawal Somya (A0326590B) · Yang Wenshuo (A0332464H) · Phua Qiuxuan (A0328352E)
Supervised by Prof Aidan Kwon · National University of Singapore

Abstract

Page 1

TraceableHands is a decentralised application (DApp) built on the Ethereum Sepolia testnet that addresses the trust deficit in charitable giving. Donors contribute USDC stablecoins to milestone-based campaigns. Funds are held in smart-contract escrow and released to vendors in two tranches: 50% automatically upon reaching the funding goal, and the remaining 50% only after a crowd-auditing process in which every donor casts a verifiable on-chain vote. Proof of delivery is stored immutably via IPFS. The system eliminates single points of failure by replacing trusted third-party auditors with decentralised donor governance. The project comprises a Solidity smart contract (OpenZeppelin-based), a React frontend, 22 automated tests, and a full Sepolia deployment.

1 Introduction

Charitable organisations globally raise hundreds of billions of dollars annually, yet a persistent lack of transparency undermines donor confidence. Studies indicate that **over 60% of potential donors cite inability to verify fund usage** as a primary barrier to giving. Existing solutions such as charity rating platforms rely on self-reported data, introducing the same trust assumptions they purport to solve.

Blockchain technology offers a foundation for *verifiable transparency*: transactions are immutable, public, and deterministically executed by code. Smart contracts can enforce fund release rules without human intermediaries. However, most existing blockchain charity projects focus only on transparent fund flows; they do not address *last-mile delivery verification* — confirming that purchased goods or services actually reached beneficiaries.

TraceableHands addresses this gap by combining:

- Milestone-based **escrow** that locks funds until conditions are met
- **IPFS content addressing** for tamper-proof delivery evidence
- **Crowd auditing** — donor-governed voting with three outcomes (Approve, Needs More Proof, Reject)

2 Problem Statement

2.1 The Donor Trust Problem

Traditional charity platforms aggregate donations and disburse funds at their discretion. This creates three failure modes:

Failure Mode	Example	Root Cause
Fund misappropriation	Funds diverted before reaching vendor	Centralised custody
Unverified delivery	Vendor reports delivery without evidence	No evidence requirement
Opaque auditing	Internal audit team with conflicts of interest	Single trusted party

2.2 Limitations of Prior Approaches

Some blockchain charity projects (e.g., BitGive, Alice.si) provide on-chain fund tracking but still rely on a *designated oracle or committee* to verify delivery. This merely shifts the trust assumption from the charity to the oracle operator. TraceableHands removes this assumption entirely by distributing audit authority across the donor pool itself.

3 System Architecture

Page 2

3.1 High-Level Architecture

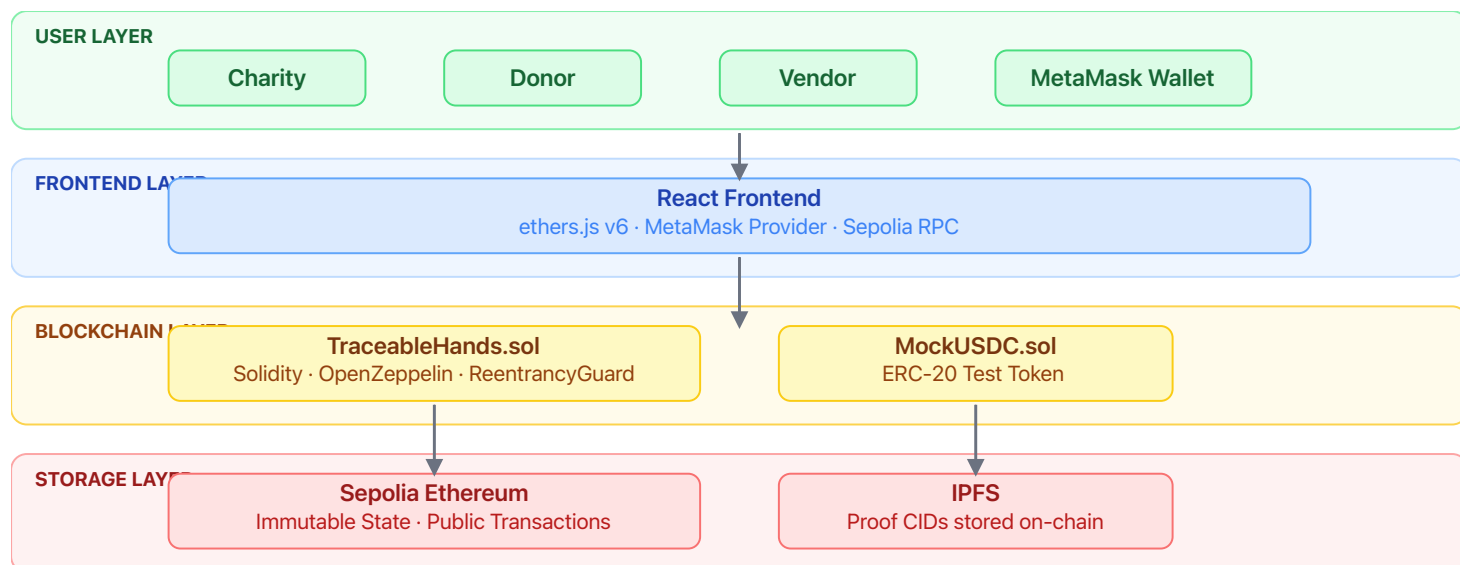


Figure 1 — TraceableHands system architecture layers

3.2 State Machine

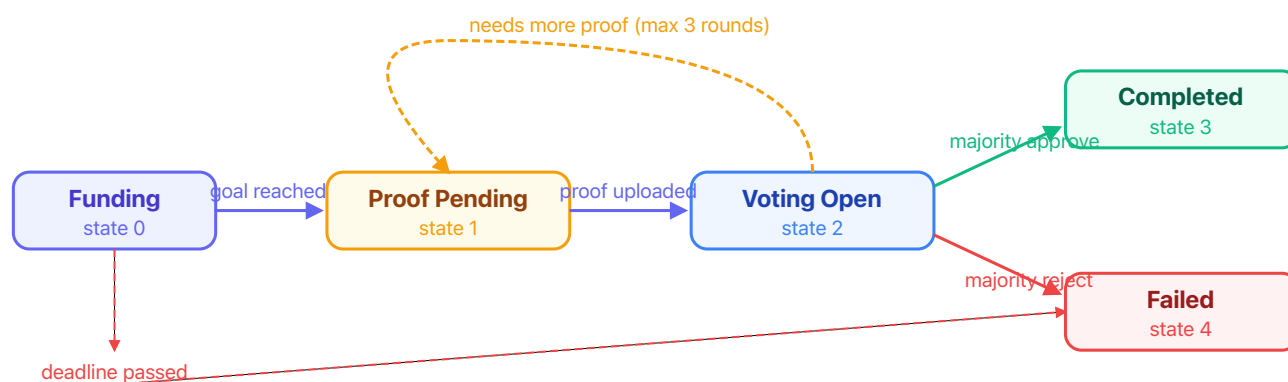


Figure 2 — TraceableHands milestone state machine (6 states)

3.3 Tech Stack Summary

Layer	Technology	Rationale
Smart Contract	Solidity ^0.8.20	Industry standard; rich tooling; overflow protection built-in post 0.8
Framework	Hardhat + vialR optimizer	Fast local EVM; vialR required for struct with many fields
Libraries	OpenZeppelin v5	Audited IERC20 + ReentrancyGuard; reduces attack surface
Frontend	React (CRA)	Component model suits per-milestone card UI
Web3 bridge	ethers.js v6	Typed contract interactions; BigNumber handling
Wallet	MetaMask	Universal browser wallet; auto network-switch API
Testnet	Sepolia	Ethereum-equivalent; stable and well-supported faucets
File storage	IPFS (Pinata pinning)	Content-addressed; CID stored on-chain for immutability
Testing	Mocha + Chai	22 tests covering all state transitions
Token	USDC (MockUSDC on testnet)	Stablecoin avoids ETH price volatility for donors

4 Smart Contract Design

4.1 Data Model

The central data structure is the `Milestone` struct. Each milestone is stored in a `mapping(uint256 => Milestone)` keyed by ID.

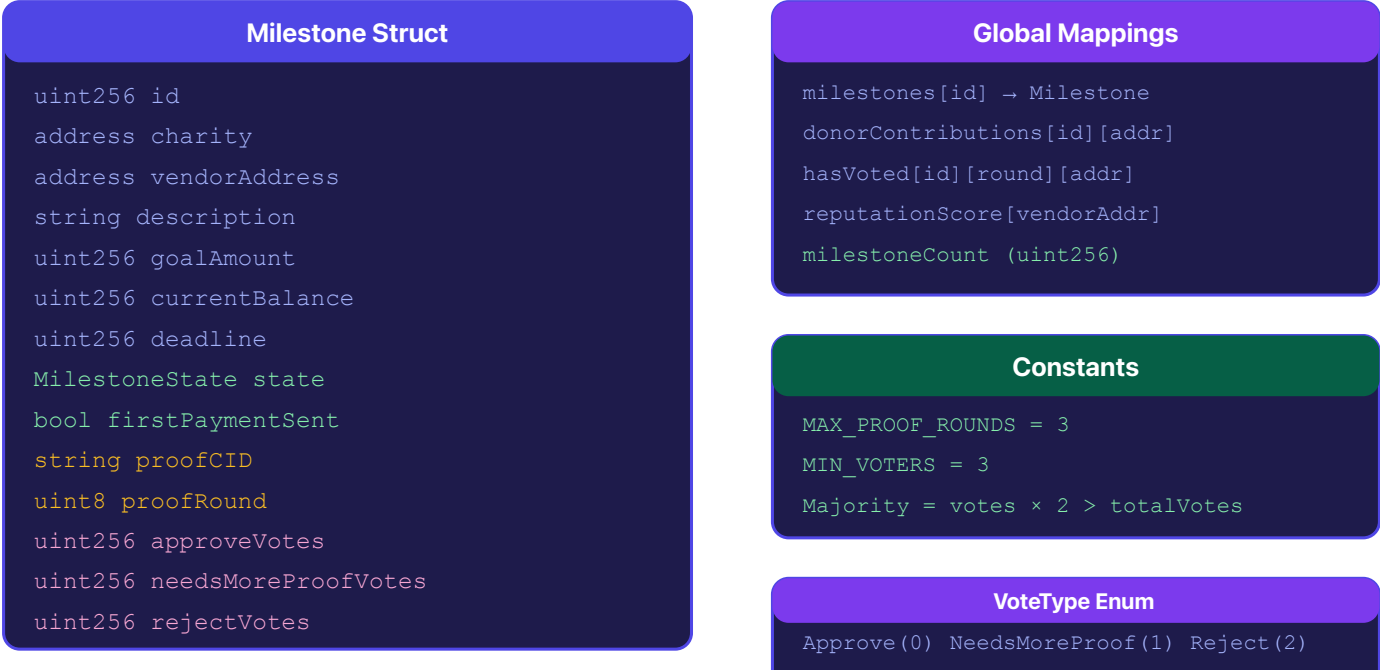


Figure 3 — Milestone data model and associated storage mappings

4.2 Key Design Decisions

Decision	Choice	Justification
Fund split	50% up-front, 50% after audit	Aligns vendor incentive: must submit proof to receive balance; donors still get partial refund if audit fails
Vote tracking key	<code>hasVoted[id]</code> <code>[round][addr]</code>	Using proof round as middle key means votes reset for free when <code>proofRound</code> increments — no expensive iteration to clear old entries
Quorum formula	<code>min(MIN_VOTERS, donorCount)</code>	Handles edge case of campaigns with fewer than 3 donors without blocking resolution indefinitely
Majority threshold	<code>votes × 2 > totalVotes</code>	Avoids floating-point division; ties never trigger an outcome, keeping voting open for more participants
Refund calculation	<code>donation / 2</code> if first payment sent	Correctly accounts for the 50% already transferred; original code had a silent bug where full-refund would fail the transfer
vialR compilation	Enabled in <code>hardhat.config.js</code>	Required to resolve "stack too deep" from Milestone struct with 15 fields

4.3 Security Properties

Threat	Mitigation
Reentrancy attack	OpenZeppelin <code>ReentrancyGuard</code> on all fund-moving functions; CEI pattern (state updated before external calls)
Sybil vote manipulation	Voters must have <code>donorContributions > 0</code> for that specific milestone — cannot vote without having locked real value
Double voting	Triple-keyed <code>hasVoted</code> mapping prevents same address voting twice per round
Proof tampering	IPFS CID is a cryptographic hash of file contents; any change produces a different CID, stored permanently on-chain
Vendor skipping proof	Milestone stays in <i>ProofPending</i> indefinitely; vendor cannot receive remaining 50% without going through voting
Integer overflow	Solidity 0.8.x built-in overflow protection; majority check uses multiplication to avoid division by zero

5 Crowd Auditing Mechanism

5.1 Design Rationale

The original design used a pre-registered *auditor pool* with reputation-gated vote thresholds. While logically sound, it introduced a new trust assumption: the auditors themselves must act honestly and in number sufficient for quorum. The crowd auditing redesign eliminates this by making **every donor an auditor for the milestones they funded**.

Property	Old Auditor Pool Design	Crowd Audit Design
Who can audit?	Registered auditor pool members	Any donor of that milestone
Skin-in-the-game	No direct financial stake	Donor's own 50% share depends on outcome
Vote options	Approve / Reject (binary)	Approve / Needs More Proof / Reject
Quorum source	Reputation-gated threshold (1–3)	$\min(3, \text{donorCount})$
Re-audit mechanism	None	Up to 3 proof rounds
Trust assumption	Auditor pool is honest	Majority of donors honest (by incentive alignment)

5.2 Voting Flow

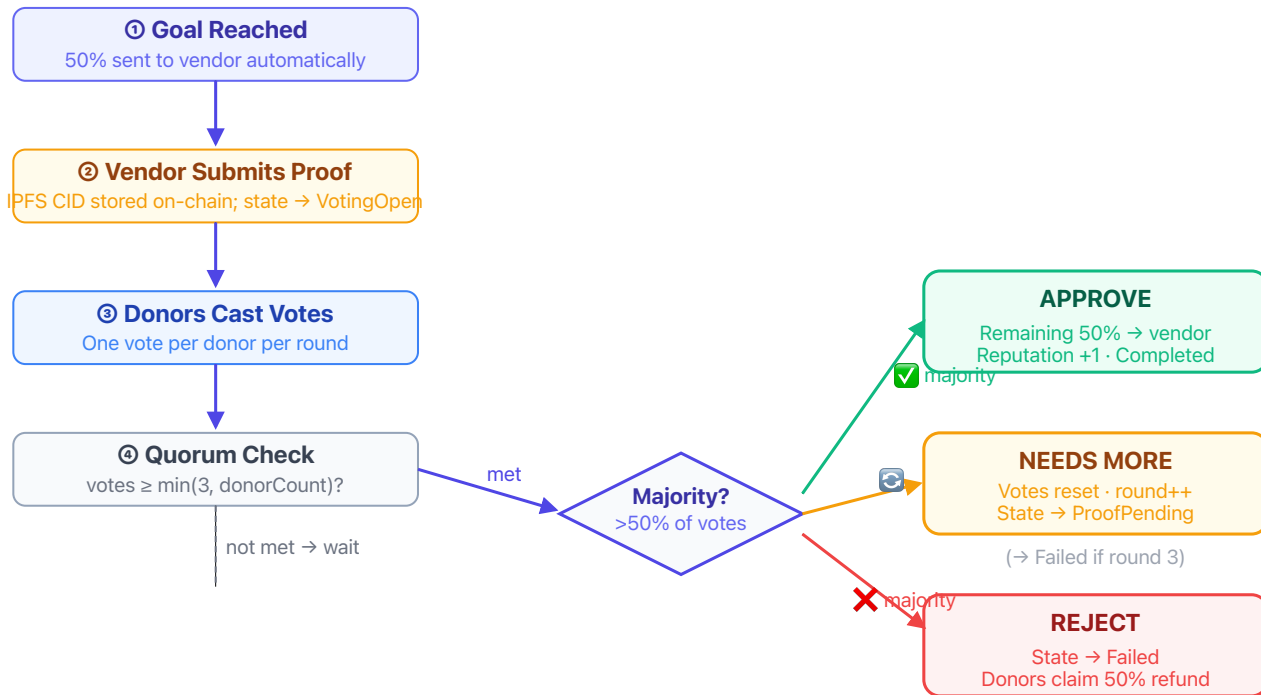


Figure 4 — Crowd voting decision flow

5.3 Incentive Alignment Analysis

The crowd auditing mechanism is incentive-compatible under rational-actor assumptions:

- **Donors voting Approve dishonestly** have no benefit — the remaining 50% goes to the vendor, not to them
- **Donors voting Reject dishonestly** recover only 50% of their donation (the other 50% already went to the vendor), making bad-faith rejection a net loss for those donors
- **Vendors submitting weak proof** risk a majority "Needs More Proof" vote, which delays payment and costs gas for re-submission
- **Vendors not submitting proof** never receive the remaining 50%, which is frozen in the contract

6 Implementation & Testing

6.1 Test Suite Overview

The project includes **22 automated tests** written in Mocha/Chai against a local Hardhat EVM. All tests pass in <500 ms.

Test Group	Tests	Coverage
Deployment	3	Token address, owner, initial state
createMilestone()	3	Correct data storage, zero-goal rejection, count increment
donate()	3	Balance update, donor count tracking, 50% split on goal
submitProof()	4	Vendor submit, charity submit, unauthorised submit, empty CID
crowdVote() — approve	2	Single-donor completion; 3-donor quorum enforcement
crowdVote() — reject	3	Majority reject fail; non-donor blocked; double-vote blocked
crowdVote() — needs more proof	2	Reset and re-submit flow; max-rounds auto-fail
claimRefund()	2	Full refund before goal; 50% partial refund after goal+fail
Total: 22 tests		100% pass ✓

✓ All 22 tests pass. The partial-refund test specifically validates the bug fix where the original contract would silently fail to transfer the full donation after 50% had already been released to the vendor.

6.2 End-to-End Demo Flow (Sepolia Testnet)

Step	Actor	Action	On-chain Effect
1	Charity	createMilestone(200 USDC, vendor, 30 days)	Milestone stored; state = Funding
2	Donor 1	approve(100) + donate(100)	donorContributions[0][D1] = 100
3	Donor 2	approve(100) + donate(100)	Goal reached → 100 USDC sent to vendor; state = ProofPending
4	Vendor	submitProof("QmABC...")	proofCID stored on-chain; state = VotingOpen
5	Donor 1	crowdVote(Approve)	approveVotes = 1; quorum not yet met (need 2)
6	Donor 2	crowdVote(Approve)	approveVotes = 2; majority met → 100 USDC sent; state = Completed; rep[vendor]++

6.3 Known Limitations & Future Work

Limitation	Proposed Enhancement
Voting has no time limit; remains open indefinitely if quorum not met	Add <code>votingDeadline</code> field; allow auto-finalize by anyone after deadline (approve if no rejection majority)
All donors have equal vote weight regardless of donation size	Weighted voting by <code>donorContributions</code> amount — more Sybil-resistant for large campaigns
No charity profile — milestones are independent	Multi-milestone charity profiles with aggregate reputation
Frontend requires localhost for multi-user testing	Deploy to Vercel/Netlify or use ngrok for shared access
IPFS pin persistence depends on Pinata free tier	Use permanent storage services (Filecoin/Arweave) for production

7 Conclusion

TraceableHands demonstrates that blockchain-based crowd auditing is a viable, incentive-compatible mechanism for verifying charitable delivery without trusted intermediaries. By giving voting rights to donors — the stakeholders with the strongest interest in accurate outcomes — the system eliminates the need for a pre-registered auditor pool while improving Sybil resistance (votes require prior financial commitment).

The three-way vote (Approve / Needs More Proof / Reject) is a meaningful improvement over binary audit systems: it allows iterative evidence refinement rather than treating every rejected first attempt as a permanent failure. Combined with the automatic partial-refund mechanism and on-chain IPFS evidence CIDs, the platform provides donors with stronger accountability guarantees than any existing centralised charity platform.

The full system — smart contract, React frontend, 22 automated tests, and Sepolia deployment — was built and tested within the scope of FT5004, demonstrating the practical viability of trustless charity infrastructure on public blockchains.

8 References

- OpenZeppelin Contracts v5 — openzeppelin.com/contracts
- Hardhat Documentation — hardhat.org
- ethers.js v6 Documentation — docs.ethers.org
- IPFS Documentation — docs.ipfs.tech
- Sepolia Etherscan — sepolia.etherscan.io
- Solidity Language Documentation — docs.soliditylang.org
- Mocha Testing Framework — mochajs.org